Workout Builder — Generated Component Structure

This document contains a ready-to-drop-in structure for your **CurrentWorkout** parent component, the isolated **ExerciseCard** child component, and the **ExercisesDialog** picker. Copy these files into your Angular project and wire imports in your module (summary below).

File: src/app/models/exercise.model.ts

```
export interface Workout {
  id: string;
  name: string;
  duration: number;
  excerciseIds?: string[];
}

export interface Exercise {
  id: string;
  name: string;
  description?: string;
  instructions: string;
  type?: ExerciseType;
  variableIds?: string[];
  primaryMuscleIds?: string[];
  secondaryMuscleIds?: string[];
  date?: Date;
  equipmentIds?: string[];
  duration?: number;
  calories?: number;
  state?: 'completed' | 'cancelled' | null;
}

export interface ExerciseType { id: string; name: string; }
export interface ExerciseVariables { id: string; name: string; extraIds?:
string[]; }
export interface Extras { id: string; name: string; }
export interface Muscle { id: string; name: string; }
export interface Equipment { id: string; name: string; }
```

File: src/app/services/firebase.service.ts

```
import { Injectable } from '@angular/core';
import { Firestore, collectionData, collection } from '@angular/fire/firestore';
import { Observable } from 'rxjs';
import { Exercise } from '../models/exercise.model';

@Injectable({ providedIn: 'root' })
export class FirebaseService {
  constructor(private firestore: Firestore) {}

  getExercises(): Observable<Exercise[]> {
    const ref = collection(this.firestore, 'exercises');
    return collectionData(ref, { idField: 'id' }) as Observable<Exercise[]>;
  }
}
```

File: src/app/current-workout/current-workout.component.ts

```
import { Component } from '@angular/core';
import { MatDialog } from '@angular/material/dialog';
import { Exercise } from '../models/exercise.model';
import { ExercisesDialogComponent } from '../exercises-dialog/exercises-
dialog.component';

@Component({
  selector: 'app-current-workout',
  templateUrl: './current-workout.component.html',
  styleUrls: ['./current-workout.component.scss']
})
export class CurrentWorkoutComponent {
  exerciseCards: Exercise[] = []; // each item produces one <app-exercise-card>

  constructor(private dialog: MatDialog) {}

  openExercisesPopup() {
    const dialogRef = this.dialog.open(ExercisesDialogComponent, {
      width: '100vw',
      height: '100vh',
      maxWidth: '100vw',
      panelClass: 'full-screen-dialog'
    });
```

```
    dialogRef.afterClosed().subscribe((result: Exercise | { custom: true } |
undefined) => {
      if (!result) return;

      if ((result as any).custom) {
        // handle opening a custom exercise flow later
        return;
      }

      // push selected exercise as a card
      const ex = result as Exercise;
      this.exerciseCards.push({ ...ex });
    });
  }

  removeCard(card: Exercise) {
    this.exerciseCards = this.exerciseCards.filter(c => c.id !== card.id);
  }
}
```

File: src/app/current-workout/current-workout.component.html

```
<div class="workout-container">
  <div class="controls">
    <button mat-raised-button color="primary" (click)="openExercisesPopup()">Add
Exercise</button>
  </div>

  <div class="cards-list">
    @for(card of exerciseCards; track card.id) {
      <app-exercise-card
        [exercise]="card"
        (remove)="removeCard(card)">
      </app-exercise-card>
    }
  </div>
</div>
```

File: src/app/current-workout/current-workout.component.scss

```scss
.workout-container {
  padding: 16px;
}

.controls {
  margin-bottom: 12px;
}

.cards-list {
  display: flex;
  flex-direction: column;
  gap: 12px;
}
```

File: src/app/exercise-card/exercise-card.component.ts

```ts
import { Component, Input, Output, EventEmitter } from '@angular/core';
import { Exercise } from '../models/exercise.model';
import { MatTableDataSource } from '@angular/material/table';

export interface ExerciseRow {
  warmup: boolean;
  weight: number;
  reps: number;
  editWeight?: boolean;
  editReps?: boolean;
}

@Component({
  selector: 'app-exercise-card',
  templateUrl: './exercise-card.component.html',
  styleUrls: ['./exercise-card.component.scss']
})
export class ExerciseCardComponent {
  @Input() exercise!: Exercise;
  @Output() remove = new EventEmitter<void>();

  displayedColumns = ['counter', 'weight', 'reps', 'actions'];
  rows: ExerciseRow[] = [];
  dataSource = new MatTableDataSource(this.rows);
```

```
  addWarmup() {
    const warmup: ExerciseRow = { warmup: true, weight: 0, reps: 0 };
    const warmups = this.rows.filter(r => r.warmup);
    const sets = this.rows.filter(r => !r.warmup);
    this.rows = [...warmups, warmup, ...sets];
    this.refresh();
  }

  addSet() {
    this.rows.push({ warmup: false, weight: 0, reps: 0 });
    this.refresh();
  }

  deleteRow(row: ExerciseRow) {
    this.rows = this.rows.filter(r => r !== row);
    this.refresh();
  }

  deleteCard() {
    this.remove.emit();
  }

  edit(row: ExerciseRow, field: 'weight' | 'reps') {
    if (field === 'weight') row.editWeight = true;
    if (field === 'reps') row.editReps = true;
  }

  stopEdit(row: ExerciseRow, field: 'weight' | 'reps') {
    if (field === 'weight') row.editWeight = false;
    if (field === 'reps') row.editReps = false;
    this.refresh();
  }

  refresh() {
    this.dataSource.data = [...this.rows];
  }

  getSetNumber(i: number) {
    return this.rows.filter(r => !r.warmup).indexOf(this.rows[i]) + 1;
  }
}
```

File: src/app/exercise-card/exercise-card.component.html

```
<mat-card class="exercise-card">
  <mat-card-header>
    <mat-card-title>{{ exercise.name }}</mat-card-title>

    <button mat-icon-button [matMenuTriggerFor]="menu">
      <mat-icon>more_vert</mat-icon>
    </button>

    <mat-menu #menu="matMenu">
      <button mat-menu-item (click)="deleteCard()">
        <mat-icon>delete</mat-icon>
        Remove
      </button>
    </mat-menu>
  </mat-card-header>

  <mat-card-content>

    <div class="center-row">
      <button mat-raised-button color="primary" (click)="addWarmup()">
        <mat-icon>add</mat-icon>
        Warm-up
      </button>
    </div>

    <table mat-table [dataSource]="dataSource" class="exercise-table">

      <!-- counter -->
      <ng-container matColumnDef="counter">
        <td mat-cell *matCellDef="let row; let i = index">
          <ng-container *ngIf="row.warmup; else normalCounter">
            <mat-icon color="accent">local_fire_department</mat-icon>
          </ng-container>
          <ng-template #normalCounter>{{ getSetNumber(i) }}</ng-template>
        </td>
      </ng-container>

      <!-- weight -->
      <ng-container matColumnDef="weight">
        <td mat-cell *matCellDef="let row">
          <div *ngIf="!row.editWeight" (click)="edit(row, 'weight')">
            {{ row.weight }} kg
          </div>
          <input *ngIf="row.editWeight" type="number" inputmode="numeric"
```

```
[(ngModel)]="row.weight" (blur)="stopEdit(row, 'weight')" />
        </td>
      </ng-container>

      <!-- reps -->
      <ng-container matColumnDef="reps">
        <td mat-cell *matCellDef="let row">
          <div *ngIf="!row.editReps" (click)="edit(row, 'reps')">
            {{ row.reps }} reps
          </div>
          <input *ngIf="row.editReps" type="number" inputmode="numeric"
[(ngModel)]="row.reps" (blur)="stopEdit(row, 'reps')" />
        </td>
      </ng-container>

      <!-- actions -->
      <ng-container matColumnDef="actions">
        <td mat-cell *matCellDef="let row">
          <button mat-icon-button [matMenuTriggerFor]="rowMenu"
(click)="setCurrentRow(row)">
            <mat-icon>more_vert</mat-icon>
          </button>
        </td>
      </ng-container>

      <tr mat-row *matRowDef="let row; columns: displayedColumns"></tr>
    </table>

    <div class="center-row">
      <button mat-raised-button color="primary" (click)="addSet()">
        <mat-icon>add</mat-icon>
        Set
      </button>
    </div>

  </mat-card-content>
</mat-card>
```

File: src/app/exercise-card/exercise-card.component.scss

```
.exercise-card { width: 100%; }
.center-row { display:flex; justify-content:center; margin: 12px 0; }
.exercise-table { width:100%; }
```

File: src/app/exercises-dialog/exercises-dialog.component.ts

```ts
import { Component, OnInit } from '@angular/core';
import { MatDialogRef } from '@angular/material/dialog';
import { FirebaseService } from '../services/firebase.service';
import { Exercise } from '../models/exercise.model';

@Component({
  selector: 'app-exercises-dialog',
  templateUrl: './exercises-dialog.component.html',
  styleUrls: ['./exercises-dialog.component.scss']
})
export class ExercisesDialogComponent implements OnInit {
  exercises: Exercise[] = [];
  loading = true;

  constructor(
    private dialogRef: MatDialogRef<ExercisesDialogComponent>,
    private fb: FirebaseService
  ) {}

  ngOnInit() {
    this.fb.getExercises().subscribe({
      next: res => { this.exercises = res; this.loading = false; },
      error: err => { console.error(err); this.loading = false; }
    });
  }

  selectExercise(ex: Exercise) {
    this.dialogRef.close(ex);
  }

  addCustomExercise() {
    this.dialogRef.close({ custom: true });
  }
}
```

File: src/app/exercises-dialog/exercises-dialog.component.html

```html
<div class="dialog-container">
  <header class="dialog-header">
    <h2>Select Exercise</h2>
  </header>
```

```
  <div class="dialog-content">
    @if (!loading) {
      <mat-list>
        @for (ex of exercises; track ex.id) {
          <mat-list-item (click)="selectExercise(ex)">
            <div mat-line>{{ ex.name }}</div>
          </mat-list-item>
        }
        @empty {
          <div class="empty">No exercises found</div>
        }
      </mat-list>
    } @else {
      <div class="loading">Loading...</div>
    }
  </div>

  <div class="dialog-bottom">
    <button mat-fab color="accent" (click)="addCustomExercise()">
      <mat-icon>add</mat-icon>
      Custom Exercise
    </button>
  </div>
</div>
```

File: src/app/exercises-dialog/exercises-dialog.component.scss

```
.dialog-container { display:flex; flex-direction:column; height:100vh; }
.dialog-header { padding:16px; border-bottom:1px solid #eee; }
.dialog-content { flex:1; overflow:auto; padding:12px; }
.dialog-bottom { position:sticky; bottom:0; padding:12px; background:#fff;
display:flex; justify-content:center; border-top:1px solid #eee; }
```

Module wiring (add to your module)

```
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { CurrentWorkoutComponent } from './current-workout/current-
workout.component';
import { ExerciseCardComponent } from './exercise-card/exercise-card.component';
```

```
import { ExercisesDialogComponent } from './exercises-dialog/exercises-
dialog.component';
import { MaterialModule } from '../material.module';
import { DragDropModule } from '@angular/cdk/drag-drop';
import { FormsModule } from '@angular/forms';

@NgModule({
  declarations: [
    CurrentWorkoutComponent,
    ExerciseCardComponent,
    ExercisesDialogComponent
  ],
  imports: [CommonModule, MaterialModule, FormsModule, DragDropModule],
  exports: [CurrentWorkoutComponent]
})
export class WorkoutModule {}
```

Notes & next steps

- The `ExerciseCardComponent` contains the table & per-card state so actions affect only that card.
- The parent `CurrentWorkoutComponent` only manages the list of cards and dialog flow.
- You can now add animations, persist cards to Firebase, or move card-level services into the child component.

If you want, I can now:

- generate the actual files as downloadable artifacts (zip)
- convert the template syntax to pure Angular HTML (no @control-flow) if you prefer
- add animations on card add/remove

Which would you like next?